

Rapport technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Branche : improve/assignment-compliance

Généré le : 2026-07-10 12:58 UTC

Data Lake financier – Démo technique

Source: /home/ubuntu/work/Projet-DataLake-Final/livrables/RAPPORT_TECHNIQUE.md

Rapport technique — Data Lake financier (version source)

1) Problème métier

Le travail demandé est de montrer une chaîne de traitement de données simple mais réaliste pour un cas financier. Les contraintes principales sont : deux sources de données, une architecture de type data lake avec au moins trois zones, une orchestration automatisable, et une API qui expose à la fois les données brutes et les données préparées.

Le sujet met aussi l'accent sur la reproductibilité. Autrement dit : les commandes doivent être relancées proprement et donner des résultats vérifiables (ou au moins des traces claires lorsqu'on ne peut pas tout faire localement).

2) Contexte technique du projet

Le dépôt est organisé autour de trois axes :

- **Ingestion** : récupérer des données brutes depuis des fichiers et une API,
- **Transformation** : enrichir les jeux de données et sortir des jeux consolidés,
- **Exposition** : fournir une API HTTP pour inspecter le pipeline.

Le projet est lancé avec Docker Compose, ce qui permet d'avoir Postgres, MinIO, Elasticsearch, Airflow et l'API dans un environnement unifié. Airflow orchestre ensuite les tâches de données.

Les composants utilisés sont :

- `ingestion/` : scripts d'import (CSV + API),
- `transformation/` : scripts de staging et curated,
- `api/` : gateway FastAPI,
- `airflow/` : DAG de planification,
- `scripts/` : benchmark et utilitaires SQL.

3) Choix d'architecture

3.1 Pourquoi trois zones

Le choix retenu est classique pour un mini data lake d'école :

- **Raw** = conservation « as-is » avec horodatage et provenance,

- **Staging** = nettoyage et normalisation,
- **Curated** = donnée enrichie pour analyse/diagnostic.

C'est une manière simple de séparer les responsabilités : on ne modifie pas directement ce qui arrive, puis on applique des transformations contrôlées, puis on produit une version prête à l'usage.

3.2 Pourquoi `MinIO` + `PostgreSQL` + `Elasticsearch`

- MinIO donne un stockage objet proche de S3, pratique pour conserver la forme brute des fichiers.
- PostgreSQL sert au stockage relationnel pour staging/curated.
- Elasticsearch garde une piste de recherche et d'accès rapide aux enregistrements bruts.

Cette combinaison est un compromis pédagogique : suffisamment réaliste pour parler d'architecture data, tout en restant testable localement.

4) Ingestion Raw : deux sources

4.1 Source fichier (CSV local)

Le module `ingestion/ingest_file.py` lit `data/finance_dataset.csv`, génère des identifiants stables via `raw_document_id`, écrit les objets dans le bucket Raw, puis indexe les données dans Elasticsearch avec métadonnées de source. La logique est volontairement orientée audit : on conserve la structure d'origine autant que possible et on ajoute `ingested_at`.

4.2 Source API (`yfinance`)

`ingestion/ingest_api.py` interroge l'API Yahoo Finance pour une liste de tickers. La sortie brute JSON est conservée en bucket Raw API, et une projection est indexée dans Elasticsearch. Les métadonnées disponibles (nom court, secteur, devise, market cap) sont ajoutées quand disponibles.

4.3 Ce que ces deux sources permettent

L'objectif n'est pas d'agrandir la couverture mondiale du marché, mais d'avoir :

- une source historique (CSV),
- une source dynamique (API),
- une logique identique de persistance brute pour comparaison.

5) Zone Staging

`transformation/staging/transform_staging.py` reprend les événements bruts et produit des séries temporelles nettoyées. Les étapes sont :

1. lecture des documents Raw dans Elasticsearch,
2. conversion des types et tri temporel,
3. déduplication par date et retrait des clôtures absentes,

4. calcul des indicateurs,
5. insertion dans `staging_ohlcv`.

Le but est d'avoir des données cohérentes par ticker avec une granularité journalière exploitable.

6) Zone Curated

`transformation/curated/transform_curated.py` prend `staging_ohlcv` et construit des indicateurs métier.

6.1 Anomalies

Le code applique Isolation Forest aux rendements, à la volatilité, au volume et au RSI. Les lignes concernées sont enregistrées avec `is_anomaly`, `anomaly_type` et `anomaly_score`. Le paramètre `contamination=0.05` influence directement le nombre d'anomalies détectées ; ce taux ne doit donc pas être présenté comme une découverte du modèle.

6.2 Signaux

Le signal combine RSI et MACD et retourne `buy`, `sell` ou `hold`. Il sert à montrer une transformation Curated ; ce n'est pas un conseil financier ni une stratégie évaluée sur données historiques.

6.3 Sortie Curated

Les résultats finaux vont vers `curated_analysis` dans PostgreSQL avec un schéma orienté consultable par API.

7) Airflow

Le DAG principal (`airflow/dags/financial_pipeline_dag.py`) orchestre la chaîne de bout en bout.

- Ingestion fichier,
- Ingestion API,
- transformation staging,
- transformation curated,
- log d'exécution.

La configuration reste simple : le DAG est planifié du lundi au vendredi à 6 h UTC. Les deux ingestions démarrent en parallèle, puis Staging et Curated s'exécutent dans l'ordre.

8) API FastAPI

Le module `api/main.py` regroupe les routes métier. Les routes exposent les zones attendues par le devoir :

- /raw et /raw/objects : lecture des données brutes,
- /staging et /staging/tickers : consultation des données préparées,
- /curated, /curated/signals, /curated/anomalies/summary : consultation analytique,
- /health, /stats : santé et métriques,
- /ingest et /ingest_fast : déclenchement d'ingestion.

Le choix d'un filtre par query parameters (ticker, limit, source, date) permet d'éviter les exports massifs et d'avoir des contrôles rapides.

9) Méthode de comparaison ``/ingest`` vs ``/ingest_fast``

Le script `scripts/benchmark_endpoints.py` exécute les deux modes d'ingestion avec des répétitions (`- - repeats`) et des tailles de batch fixes (1 et 100).

9.1 Ce que le script mesure

Pour chaque mode :

- temps d'exécution total (`wall`),
- latence médiane sur les répétitions,
- pourcentage de gain,
- statut et erreurs retournées par chaque endpoint,
- et `target_met` sur la cible fixée.

9.2 Cible d'amélioration

La cible d'amélioration demandée dans le sujet (30%) n'est pas une vérité fixe : c'est une hypothèse de test. Elle doit être recalculée sur chaque branche avec données et charge comparables. Le script écrit un JSON qui peut être relu sans dépendance à un tableau imprimé dans le README.

9.3 Pourquoi pas de chiffre figé dans la documentation

Le rapport courant évite d'annoncer une valeur unique de gain. Le benchmark doit être une action vérifiable, pas une mémoire d'un run ancien. C'est cohérent avec la règle de preuve locale actuelle : les chiffres ne sont valides que s'ils sortent de la branche courante après relance.

10) Vérification et tests

10.1 Tests hors stack

La vérification minimale locale passe par Pytest sur les modules utiles au sujet :

```
python -m pytest tests/test_financial_logic.py tests/test_benchmark.py
```

Sur cette branche, le check local est vert.

10.2 Vérification en environnement intégré

On poursuit ensuite avec Docker pour confirmer la chaîne complète :

1. `docker compose up -d`,
2. `curl /health` et `curl /stats`,
3. lecture de `/raw`, `/staging`, `/curated`,
4. exécution du benchmark.

Le point important est d'expliciter ce qui est confirmé par test unitaire et ce qui dépend d'un run Docker complet.

11) Limites constatées

- Les endpoints de santé incluent Redis pour la vérification de disponibilité, mais le coeur métier n'est pas centré sur du caching applicatif.
- Les temps et volumes dépendent de la disponibilité réseau d'yfinance.
- Les statistiques `/stats` sont exactes vis-à-vis de l'état courant des services ; elles peuvent être à réinterpréter pendant la première phase de chargement.
- Les résultats de benchmark de branche antérieure existent dans un fichier historique, mais ils ne remplacent pas un rerun propre après changements.

12) Reproductibilité

Prérequis

- Docker + Docker Compose.

Étapes

```
git clone https://github.com/Nathan2412/Projet-DataLake-Final.git
cd Projet-DataLake-Final
docker compose up -d
```

Puis :

```
curl http://localhost:8000/health
curl "http://localhost:8000/raw?limit=5"
python scripts/benchmark_endpoints.py --base-url http://localhost:8000 --period 5d --repeats 3 -
```

Le rendu final doit conserver la sortie des tests, l'état des endpoints et le JSON du benchmark relancé. Si `target_met` est faux, le rapport doit le dire au lieu de reprendre un ancien résultat.

13) État avant remise

Les tests hors stack passent. La stack Docker et le benchmark doivent encore être relancés sur cette branche. La cible `>= 30%` reste un objectif à vérifier, pas un succès garanti.